Linux commands

- cd
    - cd ~ - change working directory to home directory
    - cd - - change working directory to old working directory
    - cd .. - change working directory to parent directory
- ls
    - ls - list the contents of present working directory
    - ls path - list the contents of given path
    - ls -l - list the contents in detail format
        - type and permissions
            - Types of files
                - Regular file (-)
                - Directory file (d)
                - Link file (l)
                - pipe file (p)
                - socket file (s)
                - character special file (c)
                - block specical file (b)
            - Permissions of files
                - r - read, w - write, x - execute
                - (rwx)user/owner, (rwx)group, (rwx)others
        - link count
        - user/owner
        - group
        - size
        - timestamp
        - name
    - ls -a - list all contents along with hidden
    - ls -A - list all contents along with hidden except . and ..
    - ls -i - list contents with indoe number
        - inode number is unique number given to every file
    - ls -s - list content with size (number of blocks)
    - ls -S - list content in descending order of their sizes
- touch
    - if file does not exist, empty file is created
    - if file exist, timestamp of that file is changed
- stat
    - stat file - display information of file
    - stat file1 file2 - display information of file1 and file2
    - stat -c "format" file - display file information in given format

- head
  - head file - display first 10 lines
  - head -5 file - display first 5 lines
- tail
  - tail file - display last 10 lines
  - tail -4 file - display last 4 lines
- sort
  - sort file - sort the content by alphabetically
  - sort -n file - sort the content by their value
    - sort command do not modify file content
- uniq
  - uniq file - display contents uniquely (truncate duplicate)
    - truncate duplicate content if it is consecutive
- rev filepath
  - Print each line reversed.
  - File contents are not modified.
- tac filepath
  - Print all lines in reverse order. The first line printed at last, while last line printed first.
- stat path
  - Display info about file or directory.
- alias
  - alias list="ls -l"
    - list will be alias/nick name to ls -l
    - list will give output same as ls -l
- unalias
  - unalias list
- which
  - which command
    - display the location of command executable.
- whereis
  - whereis command
    - display the location of command executable and also manual paage location.

Types of commands

- Internal commands
  - commands are part of shell program only.
  - No seperate executable is present
  - eg. alias, unalias, cd
- External commands
  - commands are not part of shell program
  - Seperate executable is available in any one of the bin directory
  - eg. ls, cp, mv

Redirection

- for every command input is taken from terminal, output is printed on terminal and error is also printed on terminal
- Standard streams (by defualt for every process, three files are opened)
  - stdin
  - stdout
  - stderr
- There are three types of redirections
  - input redirection
    - input will be taken from file instead of stdin
    - to do input redirection '<' symbol is used
    - command < file
  - output redirection
    - output will be written into file instead of stdout
    - to do output redirection '>' or '>>' symbol is used
    - command > file
      - older content of file will be over written
    - command >> file
      - content will be appneded into file at the end
  - error redirection
    - error will be written into file instead of stderr
    - to do output redirection '2>' or '2>>' symbol is used
    - command 2> file
      - older content of file will be over written
    - command 2>> file
      - content will be appneded into file at the end

Pipe

- Using pipe, we can redirect output of any command to the input of any other command.
- Two processes are connected using pipe operator (|).

- Two processes runs simultaneously and are automatically rescheduled as data flows between them.
- If you don't use pipes, you must use several steps to do single task.
- command1 | command2
  - output of command1 will be given as input to command 2
- E.g.
  - who | wc

Shell meta characters

- '*' - zero or more occurances of any character
- '?' - one occurance of any character

File Permissions/Mode

- File permission types: r (read), w (write), x (execute)
- Permission levels: u (user), g (group), o (other)
- File mode: u-rwx g-rwx o-rwx
- Example: hello.txt -- user can read/write and execute, group can read and execute, other can only read.
  - hello.txt mode: u=rwx g=r-x o=r--
    - rwx r-x r--
    - 111 101 100 = 754
  - chmod 754 hello.txt
- Decimal and Binary
  - 000 -- 0
  - 001 -- 1
  - 010 -- 2
  - 011 -- 3
  - 100 -- 4
  - 101 -- 5
  - 110 -- 6
  - 111 -- 7
- File mode/permissions
  - read=4, write=2, execute=1
  - user -- rwx -- 4+2+1 = 7
  - group -- r-x -- 4+1 = 5
  - other -- r-- -- 4 = 4
- File mode can be changed by the file owner (or superuser).
  - rw- rw- r--
  - 110 110 100

    - 6 6 4
    - chmod 664 f1.txt
    - rw- rw- r--
    - 110 110 100
    - 111 100 100
    - 7 4 4
    - chmod 744 f1.txt
- chmod +/- r/w/x file
    - to add or remove permissions of file
- chmod u/g/o +/- r/w/x
    - to add or remove permissions of specific level

Directory permissions/Mode

- From end user perspective, directory is a container that contains subdirectories and files.
- From kernel perspective, directory is a special file (d) that contains directory entries for each subdirectory and file.
- Directory entry = File/Subdir name + Inode number
- Default directory permissions: rwx rwx r-x
- Dir read permission: Can list dir contents (i.e. can read dir entries)
- Dir write permission: Can add, delete or modify dir entries i.e. Create new subdir/file in it, Can delete subdir/file in it, Can rename subdir/file in it.
- Dir execute permission: Can changed to that directory (cd command).

Classification of OS

- OS can be categorized based on the target system (computers).
    - Mainframe systems
    - Desktop systems
    - Multi-processor (Parallel) systems
    - Distributed systems
    - Hand-held systems
    - Real-time systems

Distributed systems

- Multiple computers connected together in a close network is called as "distributed system".
- Its advantages are high availability (24x7), high scalability (many clients, huge data), fault tolerance (any computer may fail).
- The requests are redirected to the computer having less load using "load balancing" techniques.
- The set of computers connected together for a certain task is called as "cluster". Examples: Linux.

Handheld systems

- OS installed on handheld devices like mobiles, PDAs, iPODs, etc.
- Challenges:
  - Small screen size
  - Low end processors
  - Less RAM size
  - Battery powered
- Examples: Symbian, iOS, Linux, PalmOS, WindowsCE, etc.

Realtime systems

- The OS in which accuracy of results depends on accuracy of the computation as well as time duration in which results are produced, is called as "RTOS".
- If results are not produced within certain time (deadline), catastrophic effects may occur.
- These OS ensure that tasks will be completed in a definite time duration.
- Time from the arrival of interrupt till begin handling of the interrupt is called as "Interrupt Latency".
- RTOS have very small and fixed interrupt latencies.
- RTOS Examples: uC-OS, VxWorks, pSOS, RTLinux, FreeRTOS, etc.

Resident Monitor

- Early (oldest) OS resides in memory and monitor execution of the programs. If it fails, error is reported.
- OS provides hardware interfacing that can be reused by all the programs.

Batch Systems

- The batch/group of similar programs is loaded in the computer, from which OS loads one program in the memory and execute it. The programs are executed one after another.
- In this case, if any process is performing IO, CPU will wait for that process and hence not utilized efficiently.

Multi-Programming

- In multi-programming systems, multiple program can be loaded in the memory.
- The number of program that can be loaded in the memory at the same time, is called as "degree of multi-programming".
- In these systems, if one of the process is performing IO, CPU can continue execution of another program. This will increase CPU utilization.
- Each process will spend some time for CPU computation (CPU burst) and some time for IO (IO burst).
  - If CPU burst > IO burst, then process is called as "CPU bound".
  - If IO burst > CPU burst, then process is called as "IO bound".
- To efficiently utilize CPU, a good mix of CPU bound and IO bound processes should be loaded into memory. This task is performed by an unit of OS called as "Job scheduler" OR "Long term scheduler".
- If multiple programs are loaded into the RAM by job scheduler, then one of process need to be executed (dispatched) on the CPU. This selection is done by another unit of OS called as "CPU scheduler" OR "Short term scheduler".

Multi-tasking OR time-sharing

- CPU time is shared amoung multiple processes in the main memory is called as "multi-tasking".
- In such system, a small amount of CPU time is given to each process repeatedly, so that response time for any process < 1 sec.
- With this mechanism, multiple tasks (ready for execution) can execute concurrently.
- There are two types of multi-tasking:
    - Process based multitasking: Multiple independent processes are executing concurrently. Processes running on multiple processors called as "multi-processing".
    - Thread based multi-tasking OR multi-threading: Multiple parts/functions in a process are executing concurrently.

Thread concept

- Threads are used to execute multiple tasks concurrently in the same program/process.
- Thread is a light-weight process.
    - For each thread new control block and stack is created. Other sections (text, data, heap, ...) are shared with the parent process.
    - Inter-thread communication is much faster than inter-process communication.
    - Context switch between two threads in the same process is faster.
- Thread stack is used to create function activation records of the functions called/executed by the thread.

Process vs Thread

- In modern OS, process is a container holding resources required for execution, while thread is unit of execution/scheduling.
- Process holds resources like memory, open files, IPC (e.g. signal table, shared memory, pipe, etc.).
- PCB contains resources information like pid, exit status, open files, signals/ipc, memory info, etc.
- CPU time is allocated to the threads. Thread is unit of execution.
- TCB contains execution information like tid, scheduling info (priority, sched algo, time left, ...), Execution context, Kernel stack, etc.
- terminal> ps -e -o pid,nlwp,cmd
- terminal> ps -e -m -o pid,tid,nlwp

main thread

- For each process one thread is created by default called as main thread.
- The main thread executes entry-point function of the process.
- The main thread use the process stack.
- When main thread is terminated, the process is terminated.
- When a process is terminated, all threads in the process are terminated.

Multi-user

- Multiple users can execute multiple tasks concurrently on the same systems. e.g. IBM 360, UNIX, Windows Servers, etc.
- Each user can access system via different terminal.
- There are many UNIX commands to track users and terminals.
    - tty, who, who am i, whoami, w

Process

- Process is program in execution.
- Process has multiple sections i.e. text, data, rodata, heap, stack. ... into user space and its metadata is stored into kernel space in form of PCB struct.
- PCB contains
    - id, exit status,
    - scheduling info (state, priority, time left, scheduling policy, ...),
    - files info (current directory, root directory, open file descriptor table, ...),
    - memory information (base & limit, segment table, or page table),
    - ipc information (signals, ...),
    - execution context, kernel stack, ...

OS Data Structures:

- Job queue / Process table: PCBs of all processes in the system are maintained here.
- Ready queue: PCBs of all processes ready for the CPU execution and kept here.
- Waiting queue: Each IO device is associated with its waiting queue and processes waiting for that IO device will be kept in that queue

Regular Expressions

- Find a pattern in text file(s).
- Regular expressions are patterns used to match character combinations in strings.
- A regular expression pattern is composed of simple characters, or a combination of simple and special characters e.g. /abc/, /ab*c/
- Pattern is given using regex wild-card characters.
    - Basic wild-card characters
        - $ - find at the end of line.
        - ^ - find at the start of line.
        - [ ] - any single char in give range or set of chars
        - [^] - any single char not in give range or set of chars
        - . - any single character
        - zero or more occurrences of previous character
    - Extended wild-card characters
        - ? - zero or one occurrence of previous character
        - one or more occurrences of previous character
        - {n} - n occurrences of previous character
        - {,n} - max n occurrences of previous character
        - {m,} - min m occurrences of previous character
        - {m,n} - min m and max n occurrences of previous character
        - () - grouping (chars)
        - (|) - find one of the group of characters

grep

- Regex commands
    - grep - GNU Regular Expression Parser - Basic wild-card

    - egrep - Extended Grep - Basic + Extended wild-card
    - fgrep - Fixed Grep - No wild-card
- Command syntax
    - grep "pattern" filepath
    - grep [options] "pattern" filepath
        - -c : count number of occurrences
        - -v : invert the find output
        - -i : case insensitive search
        - -w : search whole words only
        - -R : search recursively in a directory
        - -n : show line number.